

Toward Discrete Drift Free Repetitive Motion Planning of Redundant Manipulators via DLCCZNN Inspired Reformulations

Yuxuan Li^a, Junpei Yang^a, Zhan Li^{a,*}

^a*School of Mechanical Engineering, Southwest Jiaotong University, Chengdu 611756,
China*

Abstract

Joint angle drift remains a key obstacle in repetitive motion planning for redundant manipulators, especially when the controller must operate in sampled data form. This paper revisits the classical drift free quadratic programming formulation from a discrete control viewpoint and develops three LCCZNN and DLCCZNN inspired routes. Method 1 and Method 2 retain the classical equality constrained shell and modify the solver layer, whereas Method 3a directly minimizes the predicted one step position error and one step drift error under dynamic joint velocity bounds. The resulting objective is strongly convex and admits a unique minimizer; when box constraints are inactive, the command is available in closed form. Offline UR3e simulations over a 2.000×10^1 s cycle show that Method 3a achieves the best accuracy among the three methods, with mean position error 1.563×10^{-7} m and final drift norm 4.667×10^{-7} rad. In 1.000×10^1 s live CoppeliaSim runs, the mean position errors are 3.044×10^{-3} m, 3.686×10^{-4} m, and 1.302×10^{-4} m for Method 1, Method 2, and Method 3a, respectively. An extended 2.000×10^1 s live run of Method 3a further reduces the mean position error to 8.398×10^{-5} m while keeping the final drift norm at 4.180×10^{-4} rad. These results provide a practical bridge from continuous time ZNN analysis to discrete robot kinematic control with explicit drift suppression.

Keywords: redundant manipulators, repetitive motion planning, drift free control, zeroing neural networks, quadratic programming, discrete time

*Corresponding author.

Email address: zhan.li@swjtu.edu.cn (Zhan Li)

1. Introduction

Redundant manipulators can use their extra degrees of freedom for trajectory tracking, joint limit avoidance, singularity avoidance, and posture regulation. In repetitive tasks, however, this redundancy also creates the well known *joint angle drift* problem: the end effector may complete a periodic path while the joints fail to return to their initial posture. Over repeated cycles, the residual drift accumulates and can eventually reduce manipulability or drive the robot toward joint limits. This issue has long motivated drift free repetitive motion planning for redundant robots [1, 2].

A standard route formulates redundancy resolution or repetitive motion planning as a constrained quadratic program solved online by recurrent neural dynamics [3, 4, 5, 6]. In the drift free setting of [1], the QP combines drift suppression, task equality, and physical bounds in a compact shell. That formulation remains important, but it is derived mainly from a continuous time neural dynamics viewpoint and is then carried into sampled control.

LCCZNN and related ZNN variants offer lower complexity solvers for time varying equations and constrained optimization [7, 8, 9, 10]. Discrete ZNN models further show that sampled implementations deserve dedicated design rather than direct transcription of continuous laws [11, 12]. What is still missing is a discrete reformulation tailored to the classical drift free repetitive motion QP.

This paper revisits that problem from the control step itself. Method 1 and Method 2 keep the classical shell and modify only the solver layer. Method 3a rewrites the objective to penalize predicted one step position and drift errors under dynamic velocity bounds. This change moves the problem from a continuous time solver view to a discrete control view.

The main contributions are as follows.

- We formulate a discrete drift free repetitive motion framework that retains the physical constraint handling of the classical repetitive motion planning literature while making the role of the sampling step explicit.
- We derive three methods under the same kinematic model and constraint logic: a sig type drift biased QP solved by a simplified primal dual neural iteration (Method 1), a positive residual energy shaped neural scheme

(Method 2), and a one step position and drift scalarized objective solved directly as a box constrained QP (Method 3a).

- We show why Method 3a is structurally advantaged: its objective is strongly convex, its minimizer is unique, and it directly optimizes the predicted next step tracking and drift energies instead of depending on the transient residual of an inner neural solver.
- We report UR3e offline simulations, quantitative live CoppeliaSim runs for all three methods, and an extended 2.000×10^1 s live validation of Method 3a. Both offline and live results identify Method 3a as the best performer.

The current manuscript reports offline numerical experiments and CoppeliaSim live runs only. Physical robot experiments are reserved for future work.

2. Related work

2.1. Drift free repetitive motion planning of redundant manipulators

Drift free repetitive motion planning emerged when redundant manipulator control moved from minimum velocity inverse kinematics to periodic end effector tasks with explicit posture recovery requirements. The representative formulation of [1] introduced a drift free criterion as a quadratic performance index and converted joint position limits into dynamic velocity bounds, yielding an online QP with hybrid constraints. Later comparative work showed that different recurrent neural solvers can lead to noticeably different online behavior for the same RMP shell [2]. This line remains the main starting point of the present paper.

2.2. Neural network solvers for QP based inverse kinematics

Recurrent neural networks have been widely studied as online solvers for inverse kinematics and redundancy resolution. Early work focused on real time computation of redundant manipulator kinematics through recurrent neural dynamics [3]. Later studies formalized redundant inverse kinematics as constrained QP problems and analyzed neural solvers with Lyapunov tools [4, 5]. A broader review is given by [6]. These studies establish the value of neural dynamics for online optimization, but they do not directly address the sampled drift free repetitive motion problem considered here.

2.3. ZNN, LCCZNN, and discrete robot control

ZNN type models are attractive for time varying problems because the update law is built from the desired error dynamics. LCCZNN reduces computational complexity while preserving strong convergence properties in dynamic equation solving [7]. Recent extensions have pushed this line toward predefined time convergence, constrained time varying QP, and robotic kinematic control [8, 9, 10]. Discrete ZNN models for robot motion generation further indicate that sampled implementations deserve separate design and analysis [11]. Our recent DLCCZNN study reaches the same conclusion for UR3e trajectory tracking [12].

2.4. Positioning of this paper

The novelty of this paper is specific rather than broad. We do not propose a new universal ZNN theory. Instead, we revisit the classical drift free repetitive motion QP through an LCCZNN and DLCCZNN inspired lens. The key step is to reformulate the *drift free QP itself* for discrete robot control rather than merely replace the neural iteration that solves it. The resulting three method comparison helps distinguish solver improvement from problem reformulation.

3. Problem formulation

3.1. Manipulator kinematics and task error

Consider a redundant manipulator with joint vector $\mathbf{q}_k \in \mathbb{R}^n$ at sampling instant k and end effector position

$$\mathbf{x}(\mathbf{q}_k) = f(\mathbf{q}_k) \in \mathbb{R}^m, \quad (1)$$

where $n > m$ and $f(\cdot)$ is the forward kinematics. Let the desired Cartesian trajectory at step k be $\mathbf{x}_d(k)$ with desired task space velocity $\dot{\mathbf{r}}_d(k)$. We define the position tracking error and joint drift error as

$$\mathbf{e}_p(k) = \mathbf{x}(\mathbf{q}_k) - \mathbf{x}_d(k), \quad (2)$$

$$\mathbf{e}_d(k) = \mathbf{q}_k - \mathbf{q}_0, \quad (3)$$

where $\mathbf{q}_0$ is the initial joint posture at the beginning of the repetitive task.

The drift free requirement is not that the joints remain fixed during the motion. On the contrary, substantial transient joint excursions are generally

necessary to execute the task. The drift free criterion concerns the *end of cycle residual*, i.e., whether the final posture returns to $\mathbf{q}_0$ after one completed period. This distinction becomes important when interpreting the experimental results later.

3.2. Discrete time prediction model

Let $\tau > 0$ be the outer control step. Using first order linearization around $\mathbf{q}_k$,

$$\mathbf{x}(\mathbf{q}_{k+1}) \approx \mathbf{x}(\mathbf{q}_k) + \tau \mathbf{J}_k \dot{\mathbf{q}}_k, \quad (4)$$

where $\mathbf{J}_k = \mathbf{J}(\mathbf{q}_k)$ is the task Jacobian and $\dot{\mathbf{q}}_k$ is the joint velocity command to be solved at step k . The joint update is

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \tau \dot{\mathbf{q}}_k. \quad (5)$$

If the desired trajectory is also discretized as

$$\mathbf{x}_d(k+1) \approx \mathbf{x}_d(k) + \tau \dot{\mathbf{x}}_d(k), \quad (6)$$

then the one step position error propagation is

$$\mathbf{e}_p(k+1) \approx \mathbf{e}_p(k) + \tau (\mathbf{J}_k \dot{\mathbf{q}}_k - \dot{\mathbf{x}}_d(k)). \quad (7)$$

Meanwhile, the drift error propagates as

$$\mathbf{e}_d(k+1) = \mathbf{e}_d(k) + \tau \dot{\mathbf{q}}_k. \quad (8)$$

3.3. Why the classical shell uses task feedback velocity

For the classical equality constrained drift free QP, a key modeling choice is whether the equality constraint uses the pure desired task velocity $\dot{\mathbf{x}}_d(k)$ or the feedback corrected quantity

$$\mathbf{b}_{fb}(k) = \dot{\mathbf{x}}_d(k) - k_p \mathbf{e}_p(k), \quad (9)$$

where $k_p > 0$ is the task feedback gain. If we impose the pure equality

$$\mathbf{J}_k \dot{\mathbf{q}}_k = \dot{\mathbf{x}}_d(k), \quad (10)$$

then (7) gives $\mathbf{e}_p(k+1) \approx \mathbf{e}_p(k)$, so the existing position error does not automatically shrink. By contrast, if we impose

$$\mathbf{J}_k \dot{\mathbf{q}}_k = \dot{\mathbf{x}}_d(k) - k_p \mathbf{e}_p(k), \quad (11)$$

then

$$\mathbf{e}_p(k+1) \approx (1 - \tau k_p) \mathbf{e}_p(k), \quad (12)$$

which makes the position correction explicit at the sampled data level. This is why the classical shells used by Method 1 and Method 2 are written with task feedback velocity rather than pure task velocity.

3.4. Dynamic velocity bounds induced by physical limits

Let the joint position limits and velocity limits be

$$\boldsymbol{\theta}_{\min} \leq \mathbf{q}_k \leq \boldsymbol{\theta}_{\max}, \quad (13)$$

$$\dot{\boldsymbol{\theta}}_{\min} \leq \dot{\mathbf{q}}_k \leq \dot{\boldsymbol{\theta}}_{\max}. \quad (14)$$

Following the drift free RMP literature [1], the position limits can be converted into dynamic velocity limits:

$$\boldsymbol{\xi}^-(k) = \max\left(\dot{\boldsymbol{\theta}}_{\min}, \frac{\eta}{\tau} (\boldsymbol{\theta}_{\min} - \mathbf{q}_k)\right), \quad (15)$$

$$\boldsymbol{\xi}^+(k) = \min\left(\dot{\boldsymbol{\theta}}_{\max}, \frac{\eta}{\tau} (\boldsymbol{\theta}_{\max} - \mathbf{q}_k)\right), \quad (16)$$

where $\eta \in (0, 1]$ is a safety shrinkage factor. These bounds guarantee that the next step joint update in (5) remains within the feasible region.

3.5. Classical drift free QP shell

Under the definitions above, the equality constrained drift free RMP shell shared by Method 1 and Method 2 can be written as

$$\begin{aligned} \min_{\dot{\mathbf{q}}_k} \quad & \frac{1}{2} \dot{\mathbf{q}}_k^\top \mathbf{W} \dot{\mathbf{q}}_k + \hat{\mathbf{c}}(k)^\top \dot{\mathbf{q}}_k \\ \text{s.t.} \quad & \mathbf{J}_k \dot{\mathbf{q}}_k = \mathbf{b}_{\text{fb}}(k), \\ & \boldsymbol{\xi}^-(k) \leq \dot{\mathbf{q}}_k \leq \boldsymbol{\xi}^+(k), \end{aligned} \quad (17)$$

where $\mathbf{W}$ is a positive definite weight matrix and $\hat{\mathbf{c}}(k)$ encodes the drift suppression bias. The differences among the three methods considered in this paper can now be stated precisely:

- Method 1 changes the construction of $\hat{\mathbf{c}}(k)$ and solves (17) by a simplified primal dual neural iteration.
- Method 2 keeps the same shell but changes the solver dynamics through positive residual energy shaping.
- Method 3a no longer uses (17); instead, it rewrites the control objective directly from (7) and (8).

4. Three drift free methods

4.1. Method 1: sig type drift biased constrained QP

Method 1 stays closest to the classical formulation in (17). Define the drift quantity

$$\Delta \mathbf{q}(k) = \mathbf{e}_d(k) = \mathbf{q}_k - \mathbf{q}_0, \quad (18)$$

and construct the drift bias term by a sig type nonlinear activation:

$$\hat{\mathbf{c}}_1(k) = \mu \phi_{\text{sig}}(\Delta \mathbf{q}(k)), \quad (19)$$

where $\mu > 0$ and the activation is applied elementwise as

$$\phi_{\text{sig}}(z_i) = \text{sign}(z_i) |z_i|^r \exp(\min(|z_i|, c_{\text{clip}})). \quad (20)$$

The resulting QP is

$$\begin{aligned} \min_{\dot{\mathbf{q}}_k} \quad & \frac{1}{2} \dot{\mathbf{q}}_k^\top \mathbf{W} \dot{\mathbf{q}}_k + \hat{\mathbf{c}}_1(k)^\top \dot{\mathbf{q}}_k \\ \text{s.t.} \quad & \mathbf{J}_k \dot{\mathbf{q}}_k = \mathbf{b}_{\text{fb}}(k), \\ & \boldsymbol{\xi}^-(k) \leq \dot{\mathbf{q}}_k \leq \boldsymbol{\xi}^+(k). \end{aligned} \quad (21)$$

To solve (21), we introduce the primal dual state

$$\mathbf{y} = \begin{bmatrix} \dot{\mathbf{q}} \\ \boldsymbol{\lambda} \end{bmatrix}, \quad (22)$$

and use a simplified LVI PDNN iteration

$$\mathbf{y}_\ell^{\text{in}} = \mathbf{y}_\ell - (\mathbf{H} \mathbf{y}_\ell + \mathbf{p}), \quad (23)$$

$$\bar{\mathbf{y}}_\ell = P_\Omega(\mathbf{y}_\ell^{\text{in}}), \quad (24)$$

$$\mathbf{y}_{\ell+1} = \mathbf{y}_\ell + \tau_s \gamma (\bar{\mathbf{y}}_\ell - \mathbf{y}_\ell), \quad (25)$$

where $P_\Omega(\cdot)$ projects the primal component onto the admissible velocity box. The final command is the primal part of the updated state.

4.2. Method 2: positive residual energy shaped neural dynamics

Method 2 keeps the same constrained QP shell and the same task feedback equality constraint as Method 1, but changes the solver dynamics. The drift bias still takes the form

$$\hat{\mathbf{c}}_2(k) = \mu \phi_{\text{sig}}(\mathbf{e}_d(k)). \quad (26)$$

The essential change occurs in the primal dual iteration. Let

$$\mathbf{r}_\ell = P_\Omega(\mathbf{y}_\ell - (\mathbf{H}\mathbf{y}_\ell + \mathbf{p})) - \mathbf{y}_\ell \quad (27)$$

be the projection residual, and define the residual energy

$$E_{\text{res}}(\ell) = \frac{1}{2} \|\mathbf{r}_\ell\|_2^2. \quad (28)$$

Then a positive nonlinear gain is introduced as

$$g(E_{\text{res}}(\ell)) = 1 + \mu \max(E_{\text{res}}(\ell), 0)^r \exp\left(\min\left(\max(E_{\text{res}}(\ell), 0), c_{\text{clip}}\right)\right). \quad (29)$$

The state update becomes

$$\mathbf{y}_{\ell+1} = \mathbf{y}_\ell + \tau_s \gamma g(E_{\text{res}}(\ell)) \mathbf{r}_\ell. \quad (30)$$

Hence, Method 2 does not change the task level optimization model; it changes how aggressively the neural solver moves toward the constrained optimum. When the residual energy is large, the update becomes more aggressive. When the residual energy is already small, (30) naturally relaxes back toward a milder correction.

4.3. Method 3a: one step position and drift scalarized objective

Method 3a is the structural departure from the previous two methods. Instead of starting from the classical equality constrained shell, it builds the control objective directly from the one step predictions (7) and (8). Using the feedback corrected position prediction

$$\mathbf{e}_p(k+1) \approx \mathbf{e}_p(k) + \tau(\mathbf{J}_k \dot{\mathbf{q}}_k - \dot{\mathbf{r}}_d(k) + k_p \mathbf{e}_p(k)), \quad (31)$$

and the drift prediction

$$\mathbf{e}_d(k+1) = \mathbf{e}_d(k) + \tau \dot{\mathbf{q}}_k, \quad (32)$$

we define the composite discrete objective

$$\begin{aligned}
\min_{\dot{\mathbf{q}}_k} \quad & \alpha \left\| \mathbf{e}_p(k) + \tau (\mathbf{J}_k \dot{\mathbf{q}}_k - \dot{\mathbf{r}}_d(k) + k_p \mathbf{e}_p(k)) \right\|_2^2 \\
& + \beta \left\| \mathbf{e}_d(k) + \tau \dot{\mathbf{q}}_k \right\|_2^2 \\
\text{s.t.} \quad & \boldsymbol{\xi}^-(k) \leq \dot{\mathbf{q}}_k \leq \boldsymbol{\xi}^+(k),
\end{aligned} \tag{33}$$

where $\alpha > 0$ and $\beta > 0$ weigh the predicted position error and predicted drift error, respectively.

Let

$$\mathbf{a}_k = \left(\frac{1}{\tau} + k_p \right) \mathbf{e}_p(k) - \dot{\mathbf{r}}_d(k). \tag{34}$$

Ignoring constant terms that do not affect the optimizer, (33) becomes

$$\begin{aligned}
\min_{\dot{\mathbf{q}}_k} \quad & \frac{1}{2} \dot{\mathbf{q}}_k^\top \mathbf{Q}_k \dot{\mathbf{q}}_k + \mathbf{c}_k^\top \dot{\mathbf{q}}_k \\
\text{s.t.} \quad & \boldsymbol{\xi}^-(k) \leq \dot{\mathbf{q}}_k \leq \boldsymbol{\xi}^+(k),
\end{aligned} \tag{35}$$

with

$$\mathbf{Q}_k = \alpha \mathbf{J}_k^\top \mathbf{J}_k + \beta \mathbf{I} + \varepsilon \mathbf{I}, \tag{36}$$

$$\mathbf{c}_k = \alpha \mathbf{J}_k^\top \mathbf{a}_k + \beta \frac{\mathbf{e}_d(k)}{\tau}, \tag{37}$$

where $\varepsilon > 0$ is a small regularization constant.

In contrast to Method 1 and Method 2, Method 3a no longer requires an equality constrained primal dual neural solve. When the box constraints are inactive, the unconstrained minimizer is available in closed form:

$$\dot{\mathbf{q}}_k^* = -\mathbf{Q}_k^{-1} \mathbf{c}_k. \tag{38}$$

The final command is obtained by clipping (38) onto $[\boldsymbol{\xi}^-(k), \boldsymbol{\xi}^+(k)]$.

4.4. Interpretation

The three methods therefore modify three different layers of the drift free control architecture.

- Method 1 changes the drift bias term in the classical QP and solves it through a simplified primal dual neural iteration.
- Method 2 keeps the same QP but changes the solver dynamics by introducing positive residual energy shaping.

- Method 3a changes the optimization problem itself, directly encoding the sampled data tracking/drift tradeoff into the control objective.

This distinction is central to the scientific message of the paper, because it makes clear that Method 3a is not simply a better tuned neural solver; it is a different discrete time control formulation.

5. Why Method 3a is structurally superior

This section does not claim an all regime global superiority theorem. Instead, it provides precise structural properties that explain why Method 3a should outperform the other two methods under the current sampled data setting.

5.1. Strong convexity and uniqueness

Define the composite objective

$$V_k(\dot{\mathbf{q}}) = \alpha \left\| \mathbf{e}_p(k) + \tau(\mathbf{J}_k \dot{\mathbf{q}} - \dot{\mathbf{r}}_d(k) + k_p \mathbf{e}_p(k)) \right\|_2^2 + \beta \left\| \mathbf{e}_d(k) + \tau \dot{\mathbf{q}} \right\|_2^2 + \varepsilon \left\| \dot{\mathbf{q}} \right\|_2^2. \quad (39)$$

Proposition 1. *If $\alpha \geq 0$, $\beta > 0$, and $\varepsilon > 0$, then $V_k(\dot{\mathbf{q}})$ in (39) is strongly convex in $\dot{\mathbf{q}}$. Consequently, the box constrained problem (35) admits a unique global minimizer for each sampling instant k .*

Proof. The Hessian of $V_k(\dot{\mathbf{q}})$ with respect to $\dot{\mathbf{q}}$ is

$$\nabla^2 V_k(\dot{\mathbf{q}}) = 2\tau^2(\alpha \mathbf{J}_k^\top \mathbf{J}_k + \beta \mathbf{I}) + 2\varepsilon \mathbf{I}. \quad (40)$$

Since $\mathbf{J}_k^\top \mathbf{J}_k$ is positive semidefinite, $\beta \mathbf{I}$ is positive definite, and $\varepsilon \mathbf{I}$ is positive definite, $\nabla^2 V_k(\dot{\mathbf{q}})$ is positive definite. Therefore, V_k is strongly convex, and the feasible set defined by the box bounds is closed and convex. A strongly convex objective over a nonempty closed convex set has a unique global minimizer. $\square$

This property is important because it gives Method 3a a unique target at every control step. In contrast, Method 1 and Method 2 solve the older equality constrained shell only approximately through a finite number of inner neural iterations, so the applied command depends not only on the QP but also on the current solver state.

5.2. Direct one step error optimality

Proposition 2. *Among all admissible joint velocities satisfying the box constraints, Method 3a selects the command that minimizes the predicted one step composite error energy in (39).*

Proof. The statement follows directly from the definition of Method 3a in (33) to (35). The optimization variable is exactly the current control command $\dot{\mathbf{q}}_k$, and the objective is exactly the weighted sum of predicted next step position error and predicted next step drift error, augmented by a small regularization term. $\square$

The significance of this proposition is conceptual as much as mathematical. Method 1 and Method 2 do *not* directly minimize the sampled data one step prediction error. Instead, they minimize a classical velocity plus drift bias objective under a task feedback equality constraint and then rely on an inner neural dynamics to approach its optimum. By construction, Method 3a is better aligned with the actual discrete control objective.

5.3. Removal of solver induced residual

For Method 1 and Method 2, let $\mathbf{r}_\ell$ denote the projection residual of the primal dual neural solver. When the inner iteration is stopped after a finite number of substeps, the realized velocity generally satisfies

$$\mathbf{J}_k \dot{\mathbf{q}}_k \approx \mathbf{b}_{\text{fb}}(k), \quad (41)$$

but not exactly, because $\mathbf{r}_\ell$ is only approximately zero. Therefore, the applied control inherits a solver state dependent residual beyond the outer sampled data approximation.

By contrast, Method 3a eliminates this inner residual source entirely. Once $\mathbf{Q}_k$ and $\mathbf{c}_k$ are assembled, the unconstrained step (38) is obtained by a direct linear solve, followed only by explicit box clipping. The difference is not merely computational; it is architectural. The command in Method 3a is computed from the control objective itself rather than from the transient state of an auxiliary neural dynamical system.

5.4. Control interpretation

The two weighted terms in Method 3a have clear physical roles. The term multiplied by α penalizes the predicted next step position tracking error, while the term multiplied by β penalizes the predicted next step posture

deviation from the cycle start. Therefore, Method 3a performs an explicit sampled data tradeoff between tracking and drift recovery. This tradeoff is easier to interpret and tune than the indirect drift bias in Method 1 or the solver energy shaping in Method 2.

Remark 1. *The propositions above explain the empirical superiority of Method 3a under the current setting, but they do not imply that it dominates every alternative in every regime. In particular, under severe saturation, model mismatch, or noisy feedback, the final performance will still depend on the Jacobian quality, sampling step, and constraint activity. The present claim is therefore deliberately limited: Method 3a is structurally better aligned with the discrete drift free control objective studied in this paper.*

6. Experimental setup

6.1. Robot model, task, and execution modes

All experiments use a six degree of freedom UR3e manipulator. The task is a heart shaped Cartesian trajectory in the YZ plane with scale factor 0.008, generated around the initial end effector position to avoid a large startup jump. Two execution modes are used:

- **Offline mode:** numerical simulation with the same UR3e kinematic model and control law over one full 2.000×10^1 s cycle.
- **Live mode:** synchronous CoppeliaSim execution through the remote API. All three methods are tested over 1.000×10^1 s, and Method 3a is further tested over 2.000×10^1 s.

The initial commanded joint configuration is

$$\mathbf{q}_0 = [0 \quad -\pi/2 \quad \pi/2 \quad 0 \quad \pi/2 \quad 0]^\top. \quad (42)$$

In offline mode, the desired heart trajectory is translated so that its start point coincides with the end effector position generated by $\mathbf{q}_0$. This removes artificial startup mismatch and makes the offline evaluation intentionally idealized.

6.2. Compared methods and parameter settings

Each method keeps its original implemented parameter setting rather than being retuned to a common gain set. All methods share the same joint bound handling, the same dynamic velocity bound rule, $\eta = 0.9$, and the same velocity limit of 2.000 rad s^{-1} . The main settings are listed in Table 1.

Table 1: Key parameter settings used in the integrated scripts.

Method	Step size τ	Main settings
Method 1	0.005	$k_p = 20, \mu = 0.1, r = 0.9, c_{\text{clip}} = 2.0, \gamma = 80.$
Method 2	0.005	$k_p = 160, \mu = 10, r = 0.9, c_{\text{clip}} = 4.0, \gamma = 1280, 20$ inner substeps.
Method 3a	0.005	$k_p = 198, \alpha = 10^4, \beta = 10^{-3}, \varepsilon = 10^{-9}.$

The values are taken directly from the integrated scripts so that the comparison reflects the implemented methods rather than a new tuning study.

6.3. Evaluation metrics

For both offline and live runs, we report the mean position error $\frac{1}{N} \sum_k \|\mathbf{x}(\mathbf{q}_k) - \mathbf{x}_d(k)\|_2$, the maximum position error, the final position error, the transient maximum absolute joint deviation $\max_{k,i} |\Delta\theta_i(k)|$, and the final drift norm $\|\mathbf{q}(T) - \mathbf{q}_0\|_2$.

The transient maximum absolute joint deviation measures how far the joints travel during execution. It is not itself the drift free criterion, which is determined by the cycle end posture residual.

6.4. Scope of the present validation

The offline results represent same model kinematic replay, in which the controller, Jacobian, and plant share the same deterministic kinematic model. The live results are obtained through the CoppeliaSim remote API and therefore include simulator servo behavior, communication effects, and feedback mismatch. The offline values should therefore be read as best case algorithmic accuracy, whereas the live values are closer to execution performance but remain simulator based rather than physical robot results.

7. Results and discussion

7.1. Quantitative offline comparison

Table 2 reports the offline results over a full 2.000×10^1 s cycle. Method 3a is clearly the best method. Relative to Method 2, it reduces the mean position error by about $200\times$ and the final drift norm by about $7.6\times$. Relative to Method 1, the reductions reach about 1.49×10^3 and more than four orders of magnitude.

Table 2: Offline accuracy comparison over a 2.000×10^1 s cycle.

Method	Mean position error (m)	Max position error (m)	Final position error (m)	Max transient $ \Delta\theta_i $ (rad)	Final drift norm (rad)
Method 1	2.333×10^{-4}	4.975×10^{-4}	1.127×10^{-4}	4.111×10^{-1}	1.034×10^{-2}
Method 2	3.121×10^{-5}	6.308×10^{-5}	6.767×10^{-7}	4.095×10^{-1}	3.553×10^{-6}
Method 3a	1.563×10^{-7}	4.047×10^{-7}	2.033×10^{-7}	4.126×10^{-1}	4.667×10^{-7}

The transient joint excursion stays close to 4.100×10^{-1} rad for all methods. This does not contradict the drift free goal. The relevant quantity is the cycle end posture residual, where Method 3a remains clearly superior.

7.2. Quantitative live comparison

Table 3 gives the 1.000×10^1 s live CoppeliaSim results. The ranking is unchanged: Method 3a is best, Method 2 is second, and Method 1 is worst. The mean position error of Method 3a is about $2.83\times$ smaller than that of Method 2 and about $23.4\times$ smaller than that of Method 1. Its final drift norm is about $10.9\times$ smaller than that of Method 2 and about $642\times$ smaller than that of Method 1.

Table 3: Live accuracy comparison over a 1.000×10^1 s CoppeliaSim run.

Method	Mean position error (m)	Max position error (m)	Final position error (m)	Max transient $ \Delta\theta_i $ (rad)	Final drift norm (rad)
Method 1	3.044×10^{-3}	5.652×10^{-3}	1.484×10^{-3}	4.508×10^{-1}	2.677×10^{-1}
Method 2	3.686×10^{-4}	6.358×10^{-4}	1.433×10^{-4}	4.105×10^{-1}	4.551×10^{-3}
Method 3a	1.302×10^{-4}	2.312×10^{-4}	5.815×10^{-5}	4.129×10^{-1}	4.170×10^{-4}

7.3. Extended live validation of Method 3a

Table 4 compares the 1.000×10^1 s and 2.000×10^1 s live runs of Method 3a. The longer cycle lowers the mean and maximum position errors by about 35.5% and 27.5%, while leaving the final drift norm almost unchanged. This indicates that a longer period mainly reduces the tracking burden rather than changing the cycle end recovery property.

Table 4: Live comparison of Method 3a over 1.000×10^1 s and 2.000×10^1 s cycles.

Duration	Mean position error (m)	Max position error (m)	Final position error (m)	Max transient $ \Delta\theta_i $ (rad)	Final drift norm (rad)
1.000×10^1 s	1.302×10^{-4}	2.312×10^{-4}	5.815×10^{-5}	4.129×10^{-1}	4.170×10^{-4}
2.000×10^1 s	8.398×10^{-5}	1.677×10^{-4}	5.837×10^{-5}	4.129×10^{-1}	4.180×10^{-4}

Method 3a: position-and-drift scalarized objective Trajectory

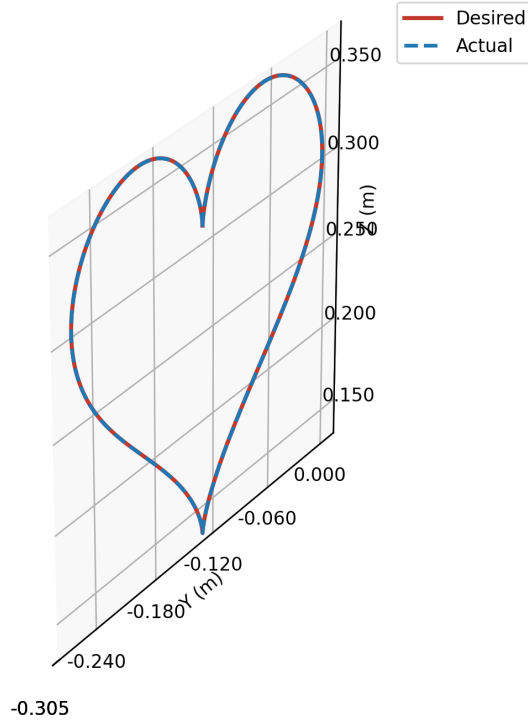


Figure 1: Extended 2.000×10^1 s live CoppeliaSim trajectory of Method 3a.

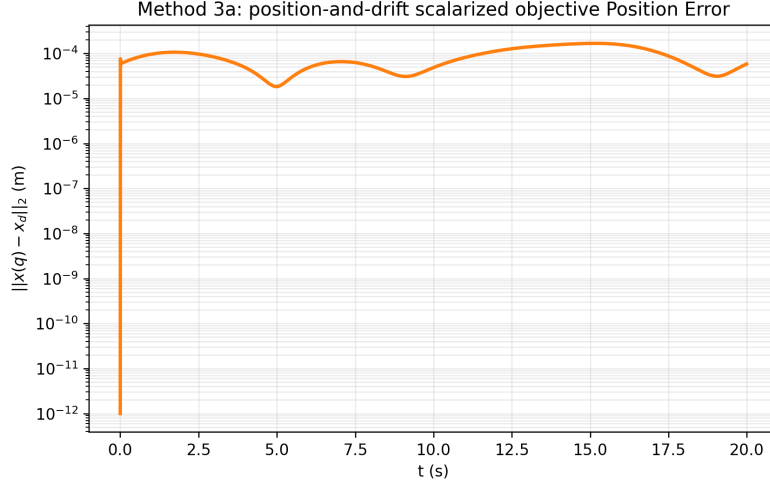


Figure 2: Extended 2.000×10^1 s live position error history of Method 3a.

The trajectory overlap in Fig. 1 remains tight, and the error in Fig. 2 stays mainly in the 10^{-5} to 10^{-4} meter range.

7.4. Offline position error traces

Figures 3 to 5 show separate offline position error histories for the three methods. Separate panels are preferable because the magnitudes differ by several orders.

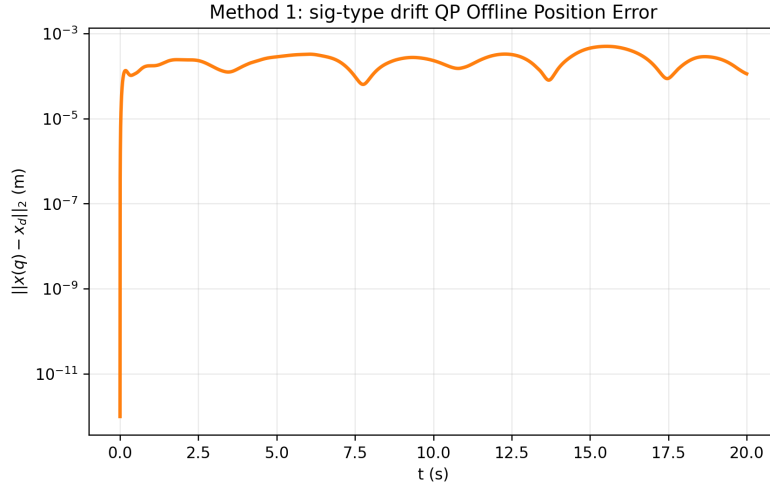


Figure 3: Offline position error history of Method 1.

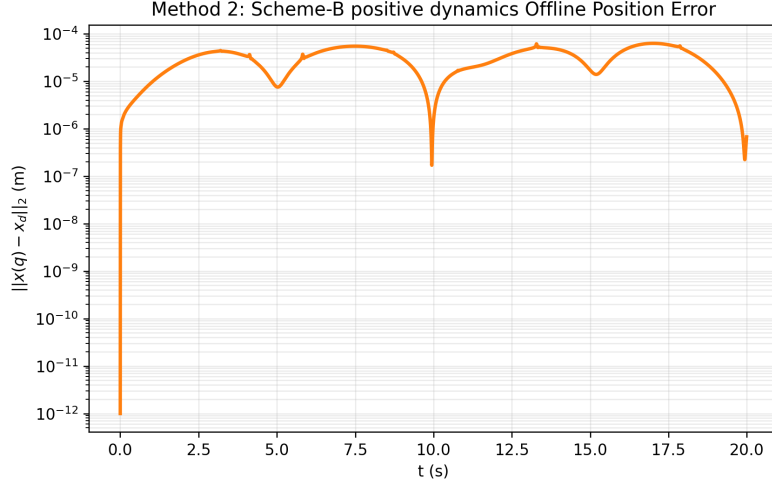


Figure 4: Offline position error history of Method 2.

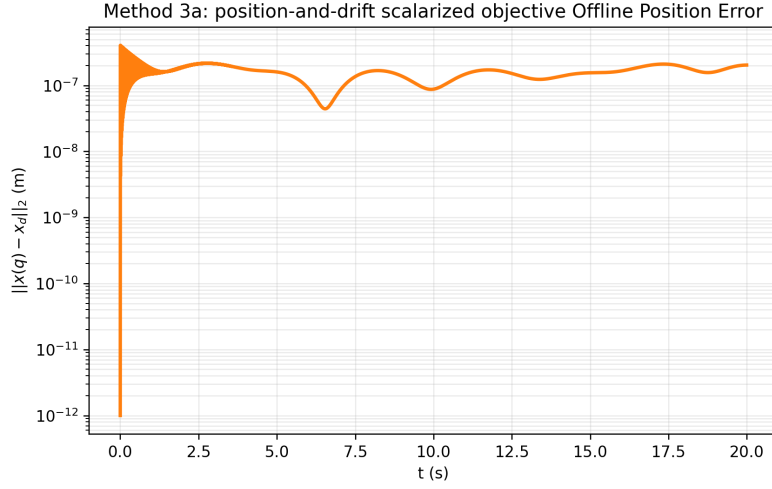


Figure 5: Offline position error history of Method 3a.

Method 1 stays in the 10^{-4} meter band, Method 2 reaches the 10^{-5} to 10^{-7} range, and Method 3a remains near the 10^{-7} meter level throughout the cycle.

7.5. End of cycle joint recovery

Tables 5 and 6 report the joint angle recovery of Method 3a after full offline and live cycles.

Table 5: Joint angle recovery of Method 3a after one 2.000×10^1 s offline cycle.

Joint	Initial angle (rad)	Final angle (rad)	Difference (rad)
θ_1	0.000000000	0.000000004	3.737×10^{-9}
θ_2	-1.570796327	-1.570796059	2.674×10^{-7}
θ_3	1.570796327	1.570796686	3.592×10^{-7}
θ_4	0.000000000	0.000000131	1.308×10^{-7}
θ_5	1.570796327	1.570796341	1.376×10^{-8}
θ_6	0.000000000	0.000000000	1.296×10^{-23}

Table 6: Joint angle recovery of Method 3a after one 2.000×10^1 s live cycle.

Joint	Initial angle (rad)	Final angle (rad)	Difference (rad)
θ_1	0.000005434	0.000004653	-7.806×10^{-7}
θ_2	-1.570523262	-1.570353389	1.699×10^{-4}
θ_3	1.570792437	1.570700526	-9.191×10^{-5}
θ_4	0.000423048	0.000792897	3.698×10^{-4}
θ_5	1.570769072	1.570744753	-2.432×10^{-5}
θ_6	0.000000000	0.000000000	0.000×10^0

Offline residuals are at the sub microradian level. In the 2.000×10^1 s live run, all six joints still return close to their initial feedback values, with the largest absolute residual equal to 3.698×10^{-4} rad at joint 4. This is consistent with the final drift norm in Table 4.

7.6. Discussion of the live and offline gap

Live errors are larger than offline errors for two main reasons. First, offline evaluation uses same model kinematic replay, whereas live execution includes remote API transmission, simulator servo behavior, and feedback mismatch. Second, the main three method live comparison uses a 1.000×10^1 s cycle, whereas the offline benchmark uses 2.000×10^1 s. The extended 2.000×10^1 s live run of Method 3a supports this interpretation, since the mean position error drops from 1.302×10^{-4} to 8.398×10^{-5} meter when the live period is doubled.

From a control perspective, Method 2 is the stronger solver variant if one wishes to preserve the classical shell with minimal reformulation. If the

goal is higher accuracy in sampled robot control, Method 3a is the more convincing route because it optimizes the discrete control objective directly.

7.7. Current limitations

The present study reports offline and CoppeliaSim live results only, so physical robot validation is still needed. In addition, only Method 3a has been tested over the full 2.000×10^1 s live cycle, and the experiments use one trajectory family and one initial posture. These points should be extended in the journal version, but they do not change the main conclusion: the strongest improvement comes from reformulating the drift free objective for discrete control rather than only replacing the solver.

8. Conclusion

This paper revisited drift free repetitive motion planning for redundant manipulators from a discrete time control perspective. Starting from the classical drift free QP framework, we derived three methods that modify different layers of the original architecture: a sig type drift biased QP with simplified primal dual neural dynamics, a positive residual energy shaped neural solver, and a one step position and drift scalarized objective solved directly at the control level.

The main conclusion is straightforward. The most effective improvement is not to reshape the old solver alone, but to rewrite the optimization problem around the sampled quantities that the controller actually regulates. This is precisely what Method 3a does. Under the current UR3e setup, it achieves the smallest offline trajectory error, the smallest offline final drift residual, and the best live performance among the three methods.

In the 1.000×10^1 s live CoppeliaSim runs, Method 3a reaches a mean position error of 1.302×10^{-4} meter and a final drift norm of 4.170×10^{-4} radian. In the extended 2.000×10^1 s live run, the mean position error is further reduced to 8.398×10^{-5} meter while the final drift norm remains at the same level. These results support the same message as the theoretical analysis: for this problem, a discrete reformulation of the drift free objective is more effective than changing only the neural solver.

Future work will add physical UR3e experiments, longer live validations for the remaining methods, and broader robustness tests.

References

- [1] Y. Zhang, Z. Tan, K. Chen, Z. Yang, X. Lv, Repetitive motion of redundant robots planned by three kinds of recurrent neural networks and illustrated with a four-link planar manipulator's straight-line example, *Robotics and Autonomous Systems* 57 (6-7) (2009) 645–651. doi:10.1016/j.robot.2009.01.002. URL <https://doi.org/10.1016/j.robot.2009.01.002>
- [2] Z. Zhang, L. Zheng, J. Yu, Y. Li, Z. Yu, Three recurrent neural networks and three numerical methods for solving a repetitive motion planning scheme of redundant robot manipulators, *IEEE/ASME Transactions on Mechatronics* 22 (3) (2017) 1423–1434. doi:10.1109/tmech.2017.2683561. URL <https://doi.org/10.1109/tmech.2017.2683561>
- [3] J. WANG, Y. ZHANG, Recurrent neural networks for real-time computation of inverse kinematics of redundant manipulators, in: *Advances in Fuzzy Systems – Applications and Theory*, WORLD SCIENTIFIC, 2004, pp. 299–319. doi:10.1142/9789812562531_0020. URL https://doi.org/10.1142/9789812562531_0020
- [4] H. Toshani, M. Farrokhi, Real-time inverse kinematics of redundant manipulators using neural networks and quadratic programming: A lyapunov-based approach, *Robotics and Autonomous Systems* 62 (6) (2014) 766–781. doi:10.1016/j.robot.2014.02.005. URL <https://doi.org/10.1016/j.robot.2014.02.005>
- [5] N. Tan, P. Yu, S. Liao, Z. Sun, Recurrent neural networks as kinematics estimator and controller for redundant manipulators subject to physical constraints, *Neural Networks* 153 (2022) 64–75. doi:10.1016/j.neunet.2022.05.021. URL <https://doi.org/10.1016/j.neunet.2022.05.021>
- [6] A. A. Hassan, M. El-Habrouk, S. Deghedie, Inverse kinematics of redundant manipulators formulated as quadratic programming optimization problem solved using recurrent neural networks: A review, *Robotica* 38 (8) (2020) 1495–1512. doi:10.1017/s0263574719001590. URL <https://doi.org/10.1017/s0263574719001590>

- [7] K. Zheng, S. Li, Y. Zhang, [Low-computational-complexity zeroing neural network model for solving systems of dynamic nonlinear equations](#), IEEE Transactions on Automatic Control 69 (7) (2024) 4368–4379. doi:[10.1109/tac.2023.3319132](#).
URL [https://doi.org/10.1109/tac.2023.3319132](#)
- [8] J. Yang, Z. Li, [A predefined-time convergent low-computational-complexity zeroing neural network for equality-constrained time-varying quadratic programming](#), Expert Systems with Applications 293 (2025) 128692. doi:[10.1016/j.eswa.2025.128692](#).
URL [https://doi.org/10.1016/j.eswa.2025.128692](#)
- [9] H. Wen, Y. Qu, X. He, S. Sun, H. Yang, T. Li, F. Zhou, [First/second-order predefined-time convergent znn models for time-varying quadratic programming and robotic manipulator application](#), ISA Transactions 146 (2024) 42–49. doi:[10.1016/j.isatra.2023.12.020](#).
URL [https://doi.org/10.1016/j.isatra.2023.12.020](#)
- [10] J. Yang, Z. Li, W. Li, [An enhanced low-computational-complexity predefined-time convergent zeroing neural network for constrained time-varying quadratic programming with kinematic control of robotic manipulator](#), IEEE Transactions on Neural Networks and Learning Systems (2026) 1–13doi:[10.1109/tnnls.2026.3672118](#).
URL [https://doi.org/10.1109/tnnls.2026.3672118](#)
- [11] B. Qiu, X.-D. Li, [Jerk-layer repetitive motion and direction control scheme of redundant robot resolved via new discretized zeroing neural network model](#), Neurocomputing 511 (2022) 237–246. doi:[10.1016/j.neucom.2022.09.015](#).
URL [https://doi.org/10.1016/j.neucom.2022.09.015](#)
- [12] Y. Li, J. Yang, Z. Li, [A discretised zeroing neural networks with low computational complexity for real time robotic kinematic control](#), local manuscript provided by the authors; cited as a preprint in preparation (2026).